СХЕМОТЕХНИКА.
ЛАБОРАТОРНАЯ РАБОТА 3.
ПРОЕКТИРОВАНИЕ ЦИФРОВЫХ СХЕМ С ПОМОЩЬЮ VERILOG.
КАФЕДРА ИВТ, КРСУ

1. Цели лабораторной работы

В последней лабораторной работе вы узнали, как использовать схематический ввод для проектирования цифровых схем в Altera Quartus II. Этапы выполненной вами работы включали: понимание постановки задачи, вывод логической функции, построение логического выражения с использованием логических элементов (НЕ, ИЛИ, И), присвоение имен входам и выходам и, наконец, использование FPGA и/или ModelSim (встроенного симулятора) для проверки схемы. Настоящая лаборатория аналогична предыдущей, но здесь используется другой подход. Вам следует написать код на Verilog для описания логического выражения вместо того, чтобы рисовать логические элементы.

2.0. Краткая информация

2.1 Verilog

Что такое Verilog? Это язык описания оборудования (HDL – Hardware Definition Language). HDL используются для выполнения логического синтеза проекта. Это означает, что мы пишем код, описывающий нашу логику, а HDL компилирует его для создания адекватной принципиальной схемы. Это упрощенное определение говорит о том, что можно просто «закодировать» нашу логику, и мы получим эквивалентную схему, не заморачиваясь ее рисованием. Verilog — это HDL, который широко используется в промышленности США с тех пор, как в 1990 году он стал открытым исходным кодом. Он является переносимым по своей природе, что означает, что один и тот же код можно использовать в различных инструментах САПР. Как и у любого языка программирования, у него есть компилятор, библиотеки, синтаксис, комментарии и т. д. Помимо Verilog существует множество HDL. Наиболее популярным среди других является VHDL (V означает очень высокоскоростная интегральная схема), и он используется в большинстве отраслей промышленности в остальном мире.

Дополнительную информацию о Verilog можно найти на сайте http://www.verilog.com/ и в учебнике, выложенном на Google Drive (папка Books).

2.2 Структурное кодирование

Когда мы смотрим на структурный уровень проекта, нас интересует, как каждая переменная обрабатывается до того, как проект выдаст результат. Например, при реализации выражения $F = A + B \cdot \overline{C}$ мы сначала **ИНВЕРТИРУЕМ** C, затем выполняем логическое **И** его с B, а затем – логическое **ИЛИ** с A.
Код на Verilog для F выглядит следующим образом:

```
module structural_example (F, A, B, C);
      input A, B, C;
      output F;          /*всегда комментируйте свой код,
                         чтобы помнить то, что вы сделали*/
      /* Этот код определяет структуру схемы */
      /* Первый символ всегда означает выходной сигнал для операций not, and и or */
      not (D,C);         // C инвертируется и сохраняется в D (выход)
      and (E,B,D);       //логическое И между B и D, результат сохраняется в E
      or (F,A,E);        // логическое ИЛИ между A и E, результат сохраняется в F

endmodule
```

2.3 Поведенческое кодирование

Другой способ создания схемы состоит в изучении ее поведенческого, а не структурного описания. Это означает, что мы смотрим на нашу логическую схему с системной точки зрения, т. е. что такое входы, что такое выходы и как они связаны друг с другом. Обычно это намного проще сделать, чем структурное описание и быстрее в смысле проектирования. Рассматривая наш предыдущий пример, мы могли бы написать следующий код:

```
module behavioral_example (F, A, B, C);
      input A, B, C;
      output F;    /*всегда комментируйте свой код,
                    чтобы помнить то, что вы сделали*/

      assign F=(A|(~C&B));      // запишите логическое выражение

endmodule
```

После этого краткого введения в Verilog мы рассмотрим два примера.

2.4. Предварительная подготовка

Создайте на устройстве, где храните свои проекты, папку **…\Lab03** и четыре подпапки **\Lab03\lab3_0**, **\Lab03\lab3_1**, **\Lab03\lab3_2** и **\Lab03\lab3_3**. В этих каталогах вы будете сохранять файлы по этой лабораторной работе.

3.0 Пример 1

Необходимо решить “**проблему фермера**”, описанную ниже, используя **Verilog**. Используйте **POS**-выражение и **SOP**-выражение для решения описанной задачи. Выполните схематическое, структурное и поведенческое описание решаемой проблемы. Цели данного лабораторного занятия:

- Научиться использовать схематическое проектирование, используя POS-выражение для решаемой задачи.
- Научиться использовать структурное описание на Verilog.
- Научиться использовать поведенческое описание на Verilog.

Описание:

У фермера есть два сарая; один к северу от усадьбы, а другой к югу. Также у него в хозяйстве есть Капуста, Коза и Волк. Каждую ночь фермер должен их размещать по сараям. Если Капуста и Коза окажутся в одном сарае, Коза съест Капусту. Если Волк и Коза будут находиться вместе, то Волк съест Козу. Вам необходимо разработать схему сигнализации, которая будет сообщать фермеру, можно ли безопасно поместить два объекта в один сарай. Таким образом, в этой схеме будет три входа: `Капуста`, `Коза`, `Волк` и один выход: `Тревога`. Выход `Тревога` показывает, что в сарае есть два объекта, которые не следует содержать вместе. Как вы уже знаете, Quartus не дружит с кириллицей, поэтому вам следует ввести новое обозначение. Например, C – Капуста (Cabbage), G – Коза (Goat), W – Волк (Wolf) и A – Тревога (Alarm).

Заполните таблицу истинности:

C	G	W	A
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Напишите POS-выражение и **SOP-** выражение для этой таблицы.

3.1 Ввод схемы с помощью Quartus Block Design Files (lab3_0) и FPGA

В этой лабораторной работе используйте схематическое проектирование и FPGA для имитации работы схемы сигнализации. Это такой же метод, который мы использовали для моделирования более простой схемы в лабораторной работе 2.

1. Проверьте свое логическое выражение для схемы сигнализации.
2. Откройте Quartus, создайте проект с именем lab3_0 и сохраните его в папке …\Lab03\lab3_0.
3. Реализуйте свое **Product-of-Sums** логическое выражение в **BDF**.
4. Скомпилируйте, назначьте выводы и запрограммируйте ПЛИС.

3.2 Структурный Verilog (lab3_1) и встроенный симулятор

В этой части лабораторной работы вы научитесь использовать структурный подход программирования на Verilog и встроенный в Quartus симулятор для симуляции схемы сигнализации.

1. Чтобы написать код Verilog, используйте текстовый редактор, генерирующий простой текст ASCII. Quartus имеет встроенный редактор, который и рекомендуется использовать. Создайте новый проект с именем lab3_1.qpf и сохраните файл в папке …\Lab03\lab3_1.
2. Затем добавьте текстовый файл, выбрав **File -> New**» в разделе **Design File**, выберите **Verilog HDL File** и сохраните его как lab3_1. v (ВАЖНО: ИСПОЛЬЗУЙТЕ ЭТО ИМЯ ДЛЯ ВАШЕГО ФАЙЛА).
3. Напишите структурное описание задачи фермера, подобное тому, что было объяснено в разделе 2.2. Опять же, вам придется использовать выражение **Product-of-Sums**. Кроме того, **имя функции вашего модуля должно совпадать с именем файла**! Это означает, что ваш модуль должен называться «module lab3_1(…);». В противном случае вы получите ошибку (см. ниже).
4. После завершения написания кода, перейдите в **Processing -> Analyze Current File**, чтобы проанализировать файл на наличие синтаксических ошибок. Если опция неактивна, убедитесь, что у вас открыт файл lab3_1.v. Если вы получили ошибки, то исправьте свой код.
5. Затем выполните симуляцию проекта (при необходимости обратитесь к предыдущим лабораторным работам за подробными инструкциями). Пытайтесь так сформировать входные сигналы так, чтобы максимальным образом проверить работоспособность вашей схемы сигнализации
6. Убедитесь в правильности работы вашего проекта.

3.3 Поведенческий Verilog (lab3_2)

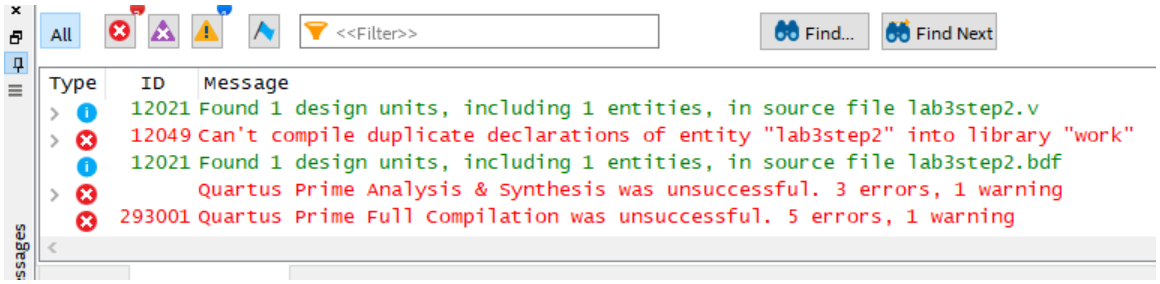
На этом этапе выполнения лабораторной работы необходимо реализовать схему сигнализации, используя Behavioral Verilog.

1. Повторите все шаги из раздела 3.2, за исключением того, что на этот раз следует использовать поведенческое описание. Создайте новый проект и назовите Verilog-файл lab3_2.v. См. раздел 2.3 для получения дополнительной информации об описании поведения.
2. Убедитесь в правильности работы вашего проекта.

3.4 Устранение неполадок

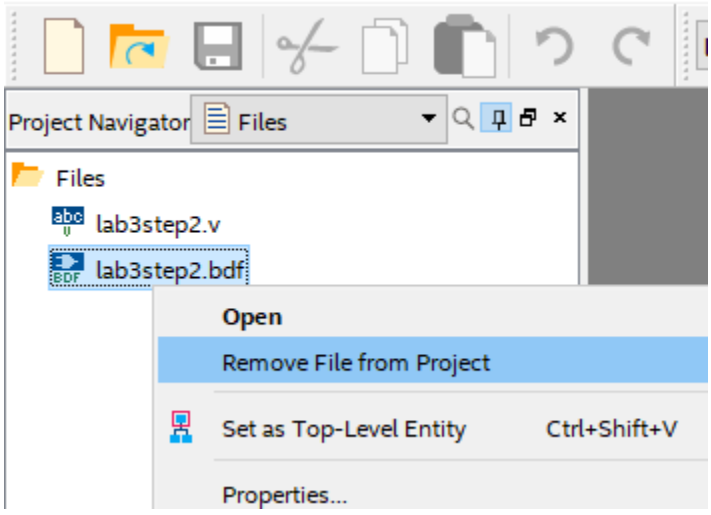
Ошибка: Can't compile duplicate declarations of entity "*filename*" into library "work" (не удается скомпилировать повторяющиеся объявления объекта **"*filename*"** в библиотеку **"work"**).

Эта ошибка обычно возникает из-за наличия двух файлов с тем же именем, что и у проекта.

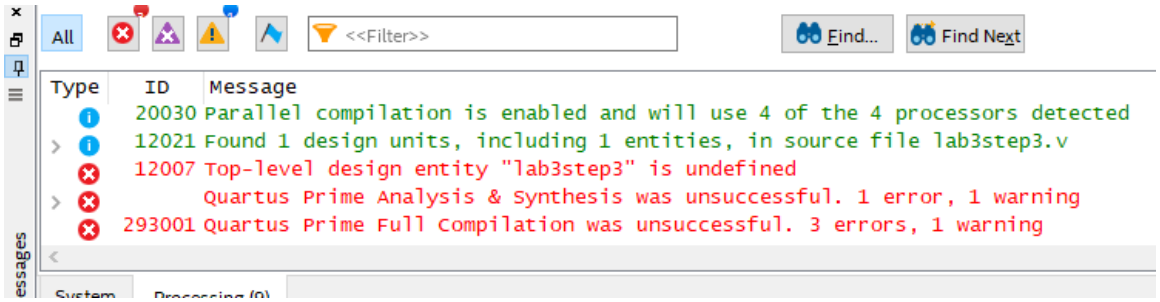


Чтобы исправить эту ошибку, вы можете:

- Создать новый проект и "Open existing file" (открыть существующий проект), чтобы перенести старый файл в новый проект.
- Перейти в окно Project Navigator в левом верхнем углу, изменить поле "Hierarchy" на просмотр "Files", щелкнуть правой кнопкой мыши на файл, который не хотите компилировать, и нажать "Remove File from Project".

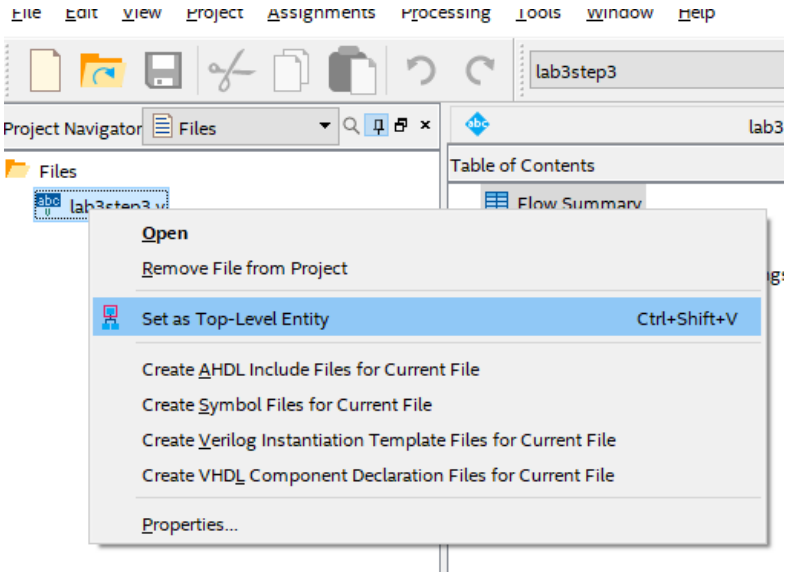


Ошибка: Top-level design entity "*filename*" is undefined (элемент верхнего уровня "***filename***" не определен).
Эта ошибка возникает из-за того, что модуль Verilog не соответствует имени файла, но также, возможно, из-за того, что файл верхнего уровня был удален, а другой не создан.



Чтобы исправить эту ошибку, вы можете:

- Переименовать модуль Verilog в соответствии с названием проекта.
- Перейти в окно Project Navigator в левом верхнем углу, измените поле "Hierarchy" на просмотр "Files", щелкнуть правой кнопкой мыши на 'основной' файл и выбрать "Set as Top-Level Entity".



4. Пример 2 (Lab3_3)
На этот раз усложним задачу фермера, добавив самого фермера в группу. Таким образом, его присутствие не даст волку съесть козу, а козе — капусту.

- Напишите каноническое **SOP**-выражение.
- Упростите это SOP-выражение и это выражение используйте в дальнейшем.
- Создайте новый проект с именем *lab3_.qpf* и сохраните файл в папке …\Lab03\lab3_3.
- Используйте Verilog для создания цифровой схемы сигнализации. Вы можете выбрать либо структурное, либо поведенческое описание.

- Проверьте правильность работы схемы на плате DE0 и с помощью встроенного симулятора.

5. Задание

5.1. Составьте таблицу истинности для решения своего варианта задания.
5.2. По таблице истинности составьте POS- и SOP-выражения.
5.3. Выполните схематическое проектирование и FPGA для реализации POS- и SOP-выражений.
5.4. Используя Verilog, выполните структурное и поведенческое описание решаемой проблемы на базе POS- и SOP- выражений.
5.5. Проверьте правильность работы схемы на плате DE0 и с помощью встроенного симулятора.

Вариант I. Перед чемпионатом по футболу Испании болельщики сделали прогноз:

- Если Барселона наберет максимальное количество очков, то Реал и Валенсия тоже наберут максимальное количество очков.
- Одновременно Барселона и Валенсия не наберут максимальное количество очков.
- Необходимым условием набора максимального количества очков Валенсией является набор максимального количества очков Реалом.

Рассмотрите три элементарных высказывания:

- Барселона наберет максимальное количество очков
- Реал наберет максимальное количество очков
- Валенсия наберет максимальное количество очков

Выведите логические выражения для каждого высказывания, базируясь на прогнозы. При возможности упростите выражение.
Составьте таблицу истинности.

Вариант II. Известно, что:

- если студент A сдаст экзамен по Схемотехнике, то и студент B тоже сдаст экзамен;
- является ложью то, что студенты B и C сдадут экзамен по Схемотехнике.

Рассмотрите три элементарных высказывания:

- A - студент A сдаст экзамен;
- B - студент B сдаст экзамен;
- C - студент C сдаст экзамен.

Выведите логические выражения для каждого высказывания, базируясь на условиях задачи. При возможности упростите выражение.
Составьте таблицу истинности.

6. Содержание отчета

6.1. Цель работы.
6.2. Схема и таблица истинности для своего варианта задания
6.3. Код на Verilog для структурного и поведенческого стиля программирования.
6.4. Выводы по работе

7. Контрольные вопросы

7.1. Заполните таблицу истинности для задачи фермера:

C	G	W	A
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

7.2. Приведите POS-выражения для этой таблицы истинности.

7.3. Напишите структурное и поведенческое описание на языке Verilog.для данного POS-выражения

7.4. Заполните таблицу истинности для расширенной задачи фермера:

F	C	G	W	A
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

7.5. Приведите SOP-выражения для расширенной таблицы истинности.
7.6. Напишите структурное и поведенческое описание на языке Verilog.для данного SOP-выражения

APPENDIX:

DE0 pin outs:

Table 4.2. Pin assignments for the pushbutton switches

Signal Name	FPGA Pin No.	Description
BUTTON [0]	PIN_ H2	Pushbutton[0]
BUTTON [1]	PIN_ G3	Pushbutton[1]
BUTTON [2]	PIN_ F1	Pushbutton[2]

Table 4.3. Pin assignments for the LEDs

Signal Name	FPGA Pin No.	Description
LEDG[0]	PIN_J1	LED Green[0]
LEDG[1]	PIN_J2	LED Green[1]
LEDG[2]	PIN_J3	LED Green[2]
LEDG[3]	PIN_H1	LED Green[3]
LEDG[4]	PIN_F2	LED Green[4]
LEDG[5]	PIN_E1	LED Green[5]
LEDG[6]	PIN_C1	LED Green[6]
LEDG[7]	PIN_C2	LED Green[7]
LEDG[8]	PIN_B2	LED Green[8]
LEDG[9]	PIN_B1	LED Green[9]

Table 4.1. Pin assignments for the slide switches

Signal Name	FPGA Pin No.	Description
SW[0]	PIN_J6	Slide Switch[0]
SW[1]	PIN_H5	Slide Switch[1]
SW[2]	PIN_H6	Slide Switch[2]
SW[3]	PIN_G4	Slide Switch[3]
SW[4]	PIN_G5	Slide Switch[4]
SW[5]	PIN_J7	Slide Switch[5]
SW[6]	PIN_H7	Slide Switch[6]
SW[7]	PIN_E3	Slide Switch[7]
SW[8]	PIN_E4	Slide Switch[8]
SW[9]	PIN_D2	Slide Switch[9]

Table 4.4. Pin assignments for the 7-segment displays.

		Description
H EX0_D[0]	PIN_EII	Seven Segment Digit 0[0]
H EX0_D[1]	PIN_F11	:2even :3egment Digit 0[1]
H EX0_D{2]	PIN_H 12	Seven :2egment Digit 0[2]
H EX0_D[3]	PIN_H 13	Seven Segment Digit 0[3]
H EX0_D[4]	PIN_G12	Seven Segment Digit 0[4]
H EX0_D[*]	PIN_F12	Seven Segment Digit 0[*]
H EX0_D[6]	PIN_F13	:2even :3egment Digit 0[6]
HEX0 DP	PIN D 13	Seven Segment Decimal P0int 0
H EX1_D[0]	PIN_A13	Seven Segment Digit 1[0]
H EX1_D[1]	PIN_613	Seven Segment Digit 1[1]
H EX1_D{2]	PIN_C13	Seven Segment Digit 1[2]
H EX1_D{3]	PIN_A14	Seven :2egment Digit 1[3]
H EX1 D[4]	PIN 614	Seven :2egment Digit 1[4]
H EX1_D[5]	PIN_E14	Seven Segment Digit 1[5]
H EX1_D[6]	PIN_A15	Seven Segment Digit 1[6]
HEX1_DP	PIN_615	Seven Segment Decimal Paint 1
H EX2_D[0]	PIN_D15	Seven :2egment Digit 2[0]
H EX2 D[1]	PIN A16	Seven :2egment Digit 2[1]
H EX2_D[2]	PIN_616	Seven Segment Digit 2[2]
H EX2_D{3]	PIN_E15	Seven Segment Digit 2[3]
H EX2_D[4]	PIN_A17	Seven Segment Digit 2[4]
H EX2_D[5]	PIN_617	Seven :2egment Digit 2[5]
H EX2_D[6]	PIN_F14	Seven :2egment Digit 2[6]
HEX2_DP	PIN_A18	Seven Segment Decimal Paint 2
H EX3_D[0]	PIN_618	Seven Segment Digit 3[0]
H EX3_D[1]	PIN_F15	Seven Segment Digit 3[1]
H EX3_D[2]	PIN_A19	Seven Segment Digit 3[2]
H EX3_D[3]	PIN_619	Seven Segment Digit 3[3]
H EX3_D[4]	PIN_C19	:3even Segment Digit 3[4]

H EX3_D[*]	PIN_D 19	Seven Segment Digit 3[5]
H EX3_D[6]	PIN_G15	Seven Segment Digit 3[6]
HEX3_DP	PIN_G16	Seven Segment Decimal Point 3